Processos de Software

Manifesto Ágil, Scrum e o início do projeto

DIM0510 — Turma 01 · Aula 02 (Sprint 0)

Prof. Fernando · UFRN · 2026.2

Roteiro de hoje

Bloco	O que vemos
Onde paramos	Do modelo prescritivo ao adaptativo
Manifesto Ágil	Valores, princípios e mal-entendidos
Origens do Scrum	De onde veio a ideia, e quando
Scrum	Pilares, papéis, artefatos e eventos
A sprint	Timebox, objetivo e compromisso
Backlog e planejamento	Histórias, critérios, estimativa e Definição de Pronto
Mãos à obra	Grupos, coorte e quadro no GitHub Projects

> No fim da aula vocês saem com grupo formado, quadro criado e a primeira sprint planejada.

Onde paramos

Na aula passada, cinco modelos de ciclo de vida e uma conclusão:

```
Cascata      decisão mais cara tomada quando se sabe menos
Modelo V     verificação por nível, mesmo problema de origem
Espiral      risco vira critério de decisão
Iterativo    feedback a cada incremento
RUP          iteração dentro da estrutura corporativa
```

Todos tentam responder à mesma pergunta: **como reduzir o tempo entre errar e descobrir?**

Onde estudar depois

Fonte	Foco
Manifesto Ágil · `agilemanifesto.org`	68 palavras, mais doze princípios
Scrum Guide · `scrumguides.org`	O documento oficial, 13 páginas
Takeuchi & Nonaka (1986)	O artigo que originou a ideia
Rubin — Essential Scrum	Livro de referência da disciplina

Parte 1

Manifesto Ágil

O que estava acontecendo em 2001

O diagnóstico

- Projetos entregues fora do prazo e do orçamento
- Requisitos aprovados em janeiro, obsoletos em julho
- Documentação extensa que ninguém lia
- Cliente ausente entre o contrato e a entrega

As respostas em curso

- Scrum, desde 1995
- Extreme Programming, 1999
- Crystal, FDD, DSDM, Adaptive SD

> Dezessete pessoas com práticas diferentes se reuniram em Utah para procurar o que havia em comum entre elas. O que saiu foi um documento de **68 palavras**.

Os quatro valores

```
Indivíduos e interações   MAIS QUE   processos e ferramentas
Software funcionando      MAIS QUE   documentação abrangente
Colaboração com cliente   MAIS QUE   negociação de contrato
Responder a mudanças      MAIS QUE   seguir um plano
```

E a frase que quase todo mundo esquece de citar:

> *"Ou seja, mesmo havendo valor nos itens à direita, valorizamos mais os itens à esquerda."*

O que o Manifesto não diz

Mito	O que o texto diz
"Ágil não tem documentação"	Documentação **abrangente** é que sai; a necessária fica
"Ágil não tem planejamento"	Responder a mudanças **não** é abrir mão do plano
"Ágil não tem processo"	Processos existem; não podem valer mais que pessoas
"Ágil não tem contrato"	Colaboração convive com contrato

> Os quatro mitos têm a mesma origem: ler só o lado esquerdo e ignorar a frase final.

Os doze princípios, em quatro grupos

Grupo	Princípios
Entrega de valor	Satisfazer o cliente com entrega contínua; entregar em semanas, não meses; software funcionando como medida de progresso
Mudança	Aceitar mudança mesmo tarde; ritmo sustentável indefinidamente
Pessoas	Construir em torno de indivíduos motivados; conversa cara a cara; equipes auto-organizadas
Excelência	Atenção à excelência técnica e ao bom design; simplicidade; retrospectiva regular

> Vale reler o grupo "Excelência". É o mais ignorado na prática, e o que mais cobra o preço depois.

Ágil não é ausência de disciplina

Sem processo	Ágil	Prescritivo
improviso	cadência curta	fases longas
sem cadência	inspeção frequente	marcos formais
sem critério	Definição de Pronto	aprovação documental

O ágil **aumenta** a frequência de decisão em vez de reduzir o rigor.

> Equipe que não tem cadência, nem critério de pronto, nem retrospectiva com consequência não é ágil. É uma equipe sem processo que usa vocabulário ágil.

Parte 2

Origens do Scrum

A ideia é anterior ao software.

Corrida de revezamento

> O estilo de "corrida de revezamento" aplicado ao desenvolvimento de produtos pode conflitar com os objetivos de velocidade e flexibilidade máximas.

Cada especialidade termina sua etapa e passa o bastão adiante: requisitos, projeto, código, teste.

O problema não é a divisão do trabalho. É que **o aprendizado de cada etapa chega tarde demais** para as anteriores — quem testa descobre coisas que quem levantou requisitos precisaria saber.

Takeuchi & Nonaka, *The New New Product Development Game*, Harvard Business Review, 1986.

Ou um time de rugby

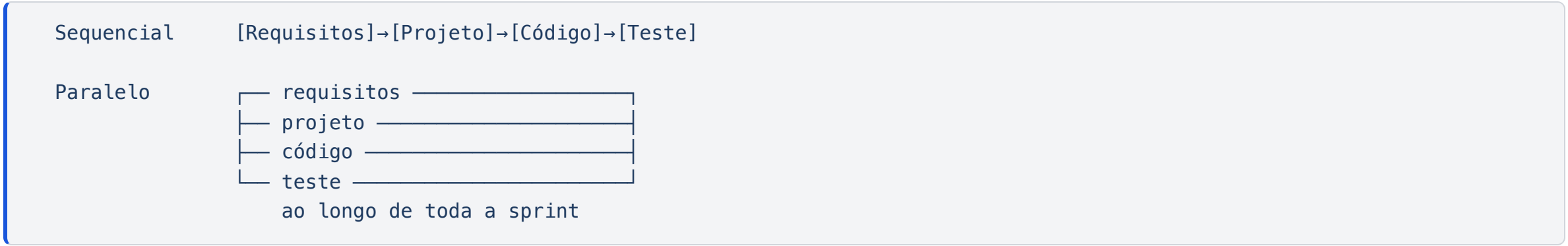
> Um estilo holístico, em que a equipe busca, como num jogo de rugby, chegar de forma integrada ao gol, pode servir melhor às necessidades competitivas atuais.

O artigo relata experiências de equipes multidisciplinares em Fuji-Xerox, Canon, Honda, Epson, 3M e HP.

A imagem do *scrum* — a formação em que o time avança junto — deu nome ao framework quase dez anos depois.

Takeuchi & Nonaka, *The New New Product Development Game*, Harvard Business Review, 1986.

Sequencial e paralelo



Em vez de completar uma coisa por vez, a equipe faz **um pouco de cada coisa, o tempo todo**.

A consequência é que o produto está sempre em um estado demonstrável, e não apenas ao final.

Linha do tempo

Quando	O quê
1986	Artigo de Takeuchi e Nonaka
1993	Primeiro uso do Scrum por Jeff Sutherland, na Easel
1995	Apresentação de Schwaber e Sutherland na OOPSLA
2001	Manifesto Ágil
2002	Fundação da Scrum Alliance
2020	Edição vigente do Scrum Guide

O Scrum é anterior ao Manifesto Ágil, e foi uma das práticas que o motivaram — não o contrário.

Parte 3

Scrum

Pilares, papéis, artefatos e eventos.

Scrum: o que é, e o que não é

> **Scrum** é um *framework* leve para desenvolver produtos em ambientes complexos, apoiado em empirismo: conhecimento vem da experiência, e decisões se tomam com base no que se observa.

É: um conjunto mínimo de papéis, eventos, artefatos e regras que se conectam.

Não é: metodologia com receita, nem prática de engenharia. Scrum não diz como testar, como versionar, nem como projetar — isso vem do XP, que vemos na próxima aula.

> Scrum é deliberadamente incompleto. Preencher as lacunas é trabalho da equipe, e é o que vocês vão registrar no acordo de processo.

O que caracteriza o Scrum

- Equipes que **se auto-organizam**
- **Timebox**: duração fixa para a sprint e para cada cerimônia
- O produto evolui em uma série de sprints
- Os itens de trabalho ficam num **Product Backlog** ordenado
- **Não há práticas de engenharia prescritas**

O último ponto costuma surpreender: o Scrum não diz nada sobre testes automatizados, integração contínua ou revisão de código.

Ele organiza **quando** as decisões acontecem e **quem** as toma. O como é escolha da equipe.

Os três pilares

Pilar	O que exige	Como aparece no Scrum
Transparência	O trabalho e seu estado visíveis a quem decide	Quadro atualizado, backlog público
Inspeção	Verificação frequente dos artefatos e do progresso	Daily, Review, Retrospectiva
Adaptação	Ajustar assim que um desvio é detectado	Replanejamento na Sprint, ações da retrospectiva

> Os três se sustentam em sequência: sem transparência a inspeção é ficção, e sem inspeção a adaptação vira palpite.

Os três papéis

Papel	Responsabilidade central
Product Owner	Maximizar o valor do produto. É quem **ordena** o backlog, e decide sozinho o que entra
Scrum Master	Garantir que o Scrum seja entendido e praticado. Remove impedimentos, não distribui tarefas
Developers	Quem constrói o incremento. Decidem **como** fazer, estimam e se auto-organizam

Papel não é cargo: uma pessoa pode acumular papéis, e no projeto de vocês isso vai acontecer.

> O Scrum Master **não** é gerente de projeto, e o Product Owner **não** é o cliente — é quem representa o cliente e responde pelas prioridades.

Product Owner, em detalhe

Responsabilidade	Detalhe
Define as funcionalidades	E, principalmente, o que fica **fora**
Ordena o Product Backlog	Por valor, não por ordem de chegada
Decide conteúdo e data de entrega	—
Responde pelo retorno do investimento	—
Aceita ou recusa o resultado	Ao final de cada sprint

É um papel de **decisão**, não de intermediação. Um Product Owner que precisa consultar outra pessoa para ordenar o backlog não está exercendo o papel.

Developers e auto-organização

- Tipicamente de **5 a 10 pessoas**, multidisciplinar
- Responde coletivamente pelo incremento
- Cada pessoa **escolhe** o que puxar; não há atribuição vinda de fora
- Mudanças de composição acontecem **entre** sprints, não durante

Auto-organizar-se diz respeito ao **como**, não ao **o quê**, e não significa ausência de prazo.

Nesta disciplina, as equipes são de **1 a 4 pessoas**, e os três papéis se acumulam. Vale saber que isso é uma adaptação, e não o Scrum como descrito no guia.

Os artefatos e seus compromissos

Artefato	O que é	Compromisso
Product Backlog	Lista ordenada de tudo o que pode ser feito	**Product Goal** — o objetivo de longo prazo
Sprint Backlog	O que a equipe escolheu para esta sprint, e o plano	**Sprint Goal** — o objetivo desta sprint
Increment	O que ficou pronto e utilizável ao fim da sprint	**Definição de Pronto**

> Cada artefato carrega um compromisso que o torna verificável. Backlog sem objetivo vira lista de desejos; incremento sem Definição de Pronto vira "quase pronto".

Os cinco eventos



Evento	Pergunta que responde
Planning	Por que esta sprint tem valor? O que entra? Como será feito?
Daily	O que impede o Sprint Goal hoje?
Review	O incremento serve? O que muda no backlog?
Retrospectiva	O que no **nosso processo** vamos mudar?

Planning: duas metades

Metade	O que acontece	Saída
Priorização	Analisar o backlog ordenado e escolher o foco	**Sprint Goal**
Planejamento	Decidir como alcançá-lo e quebrar em tarefas	**Sprint Backlog**

Entradas consideradas: o Product Backlog, o produto atual, a capacidade da equipe, as condições de negócio e a tecnologia disponível.

As tarefas são identificadas **de forma colaborativa** — não pelo Scrum Master, nem pelo Product Owner.

Do item à tarefa

```
Como agente de viagens, quero ver          Camada de negócios      (8h)
as fotos dos hotéis disponíveis      →     Interface do usuário    (4h)
                                           Testes                  (4h)
                                           Testes de desempenho    (4h)
```

O item do backlog descreve um **resultado para alguém**. As tarefas descrevem o **trabalho** para chegar lá.

Tarefas costumam ser estimadas em horas, tipicamente entre 1 e 16. Uma tarefa maior que isso geralmente ainda não foi compreendida.

Reunião diária

Parâmetro	Valor
Frequência	Diária
Duração	15 minutos
Formato	Em pé, no mesmo horário

Três perguntas, numa das formas possíveis de conduzi-la:

1. O que fiz desde ontem?
2. O que farei hoje?
3. O que está me atrapalhando?

Não é para resolver problemas. Impedimentos são registrados e tratados depois. Qualquer pessoa pode assistir; apenas a equipe fala.

Review e Retrospectiva

	Review	Retrospectiva
Olha para	O **produto**	O **processo**
Quem participa	Equipe e convidados	Toda a equipe
Duração	~2 h de preparação, sem slides	15 a 30 minutos
Saída	Backlog reordenado	Uma ação de melhoria

Na Review, a equipe **demonstra** o software em execução, e o Product Owner aceita ou recusa os itens.

Uma forma de conduzir a Retrospectiva é discutir o que a equipe gostaria de **começar**, **parar** e **continuar** fazendo.

> Nesta disciplina, pedimos **uma** mudança de processo por retrospectiva. Muitas ações simultâneas costumam resultar em nenhuma efetivada.

Erros que aparecem sempre

Erro	O que acontece
Daily como relatório de status	Vira prestação de contas ao professor ou ao chefe, e para de servir à equipe
Product Owner ausente	A equipe decide prioridade por conta própria e o cliente reclama no fim
Sprint sem objetivo	Vira lista de tarefas soltas; não há como dizer se a sprint deu certo
Retrospectiva sem ação	Reclama-se do mesmo problema em toda sprint
"Pronto" sem definição	O incremento acumula dívida invisível

> Neste curso, a **retrospectiva com ação registrada** é entregável. É o item que mais separa grupo que amadurece de grupo que repete o mesmo erro.

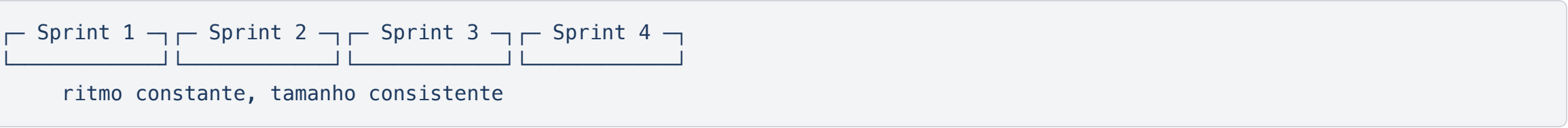
Parte 4

A sprint

Timebox, objetivo e compromisso.

O que é uma sprint

- Duração **fixa** — de uma semana a um mês
- Curtas e **consistentes** entre si
- O objetivo não muda depois de iniciada
- Termina num estado definido pela **Definição de Pronto** da equipe



Idealmente, cada sprint produz uma versão **potencialmente entregável** — pronta para ser liberada, mesmo que a decisão seja não liberar.

Por que timebox

Efeito	Como
Limita o trabalho em andamento	Não cabe mais do que o ciclo comporta
Força priorização	Se não cabe tudo, algo precisa ficar de fora
Demonstra progresso	Há sempre algo a mostrar na data
Evita perfeccionismo	O prazo encerra a discussão
Motiva o fechamento	Terminar vale mais que começar
Melhora a previsibilidade	O ritmo se torna mensurável

O prazo fixo transforma escopo em variável de decisão. Sem ele, escopo e prazo crescem juntos.

Por que iterações curtas

Benefício	Consequência
Planejamento mais simples	Menos incerteza acumulada
Retorno rápido	O erro aparece em semanas, não em meses
Erros limitados	O prejuízo cabe numa sprint
Pontos de verificação frequentes	Cadência e ritmo
Velocidade conhecida	A equipe aprende quanto cabe

Quanto mais longo o ciclo, mais cara fica a descoberta de que a direção estava errada.

Sprint Goal

Uma frase curta que descreve o **propósito** da sprint, não a lista de tarefas.

Exemplos
Suportar a geração inicial de relatórios
Carregar e organizar os dados de mapas do Brasil
Demonstrar o envio de mensagem de texto pela pilha completa
Conseguir uma impressão básica funcional, com busca por data

O objetivo permite decidir, no meio da sprint, o que fazer quando algo não couber: mantém-se o que serve ao objetivo.

Um conjunto de itens sem objetivo comum é uma lista de tarefas, não uma sprint.

Compromisso mútuo

Quem	Compromete-se a
Developers	Atender ao objetivo até o fim da sprint
Product Owner	Não alterar o objetivo durante a sprint

É possível **esclarecer** o objetivo; mudá-lo, não.

Itens do backlog podem não estar totalmente detalhados no início, e é normal que a equipe faça perguntas ao Product Owner ao longo do ciclo — a ordenação de uma lista, o formato de um campo, um caso de borda.

A diferença entre esclarecer e mudar é se o propósito da sprint continua o mesmo.

Sem mudanças durante a sprint

Mudança é qualquer alteração de trabalho ou de recursos com potencial de gerar desperdício, interromper o fluxo ou aumentar o escopo.

Exemplos	
Acrescentar uma funcionalidade nova	Interrompe o fluxo
Remover um item já iniciado	Descarta trabalho feito
Realocar pessoas	Desfaz o compromisso do planejamento

Planeje a duração da sprint de acordo com o tempo pelo qual a equipe **consegue se comprometer a não mudar**.

> Se uma semana já é difícil, o problema não é a duração: é a estabilidade do contexto.

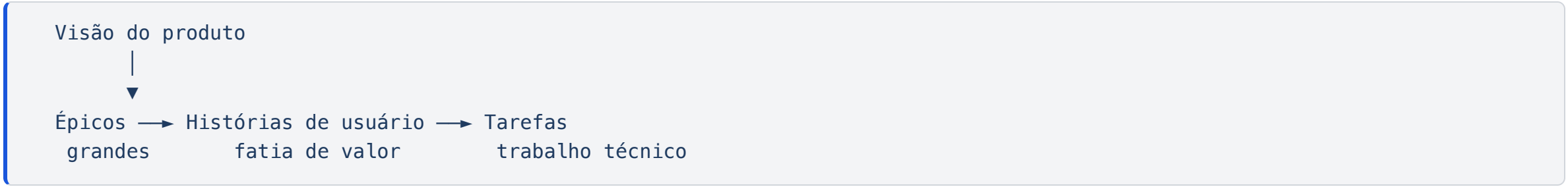
Parte 5

Backlog e planejamento

Do produto à história, e da história ao pronto.

Do produto ao backlog

O backlog não nasce de uma lista de telas. Nasce da visão:



Cada história é uma **fatia vertical**: chega ao usuário, mesmo que estreita.

> "Criar o banco de dados" não é história de usuário — é tarefa. O usuário não percebe nada quando ela termina.

Product Backlog

- Lista **ordenada** de tudo que se deseja no produto
- Ordenada pelo Product Owner
- Reordenada no início de cada sprint
- Nunca está completa nem congelada

Não é um documento de requisitos: é uma lista viva, em que a posição de cada item carrega uma decisão sobre valor e sobre quando será feito.

> Uma lista sem ordem é um repositório de ideias, e não um backlog.

Tipos de item de backlog

Tipo	Exemplo
Funcionalidade	Como atendente, quero abrir um chamado para registrar o pedido
Mudança	Ordenar resultados por sobrenome, e não por número do chamado
Defeito	Caracteres especiais na busca derrubam a consulta
Melhoria técnica	Atualizar para a versão mais recente do banco
Aquisição de conhecimento	Prototipar duas arquiteturas e medir qual atende melhor

O último tipo é o mais esquecido, e é o que permite tratar incerteza como trabalho planejado — em vez de algo que se faz por fora do quadro.

> Nem todo item é história de usuário, e forçar o formato `Como… Quero… Para…` num defeito costuma piorar o texto.

História de usuário

```
Como [papel]
Quero [ação]
Para [benefício]
```

Exemplo:

```
Como estudante do grupo
Quero registrar quanto cada um gastou numa despesa compartilhada
Para saber quem deve a quem sem refazer a conta no papel
```

> A terceira linha é a que costuma faltar, e é a mais importante: é ela que permite ao Product Owner priorizar, e à equipe propor uma solução diferente da que foi pedida.

INVEST: como saber se a história está boa

Letra	Critério	Sinal de problema
I	Independente	Só pode ser feita depois de outras três
N	Negociável	Já vem com a solução técnica embutida
V	Valiosa	O usuário não percebe diferença
E	Estimável	Ninguém consegue dizer o tamanho
S	Pequena	Não cabe numa sprint
T	Testável	Não dá para dizer se está pronta

> Falhar em **E** e **T** quase sempre significa a mesma coisa: a história ainda não foi entendida.

Critérios de aceitação

O que precisa ser verdade para a história estar pronta. Escreva antes de implementar.

```
História: registrar quanto cada um gastou numa despesa compartilhada

Critérios de aceitação
  – Valor negativo ou zero é recusado, com mensagem
  – A despesa aparece na lista do grupo imediatamente após salvar
  – O saldo de cada participante é recalculado
  – Despesa sem participante selecionado não pode ser salva
```

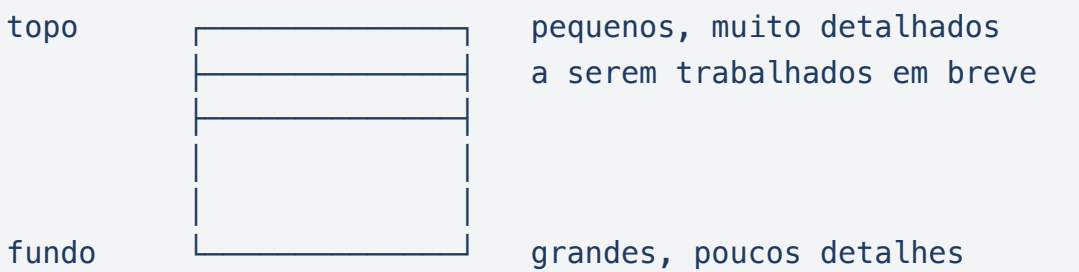
> Critério de aceitação é o rascunho do teste. Se você não consegue escrever o critério, também não vai conseguir escrever o teste — e provavelmente não entendeu a história.

Um bom backlog é DEEP

Letra	Significa
Detalhado adequadamente	Itens próximos, detalhados; distantes, genéricos
Emergente	Muda continuamente; nunca congela
Estimado	Cada item tem uma estimativa de tamanho
Priorizado	Ordenado, com mais esforço de ordenação no topo

Rubin, *Essential Scrum*, cap. 6.

Detalhamento progressivo



Ao se aproximar de um item grande, ele é dividido em itens menores e prontos para entrar numa sprint.

O detalhamento não deve acontecer **cedo demais**, porque o contexto ainda vai mudar, nem **tarde demais**, porque a sprint começa sem clareza. Os dois extremos geram desperdício.

Refinamento do backlog

Atividade contínua, não um evento com data marcada. Cerca de 10% do tempo da equipe.

O que acontece	Resultado
Quebrar épicos em histórias	Itens que cabem numa sprint
Escrever critérios de aceitação	Itens testáveis
Estimar itens ainda sem tamanho	Itens estimáveis
Reordenar conforme o que se aprendeu	Topo do backlog confiável

O objetivo é chegar ao Planning com os itens do topo já compreendidos.

> Um Planning que gasta a hora inteira entendendo histórias é sinal de que o refinamento não aconteceu — e a sprint começa com meia hora a menos.

Estimativa em pontos

Pontos medem **tamanho relativo**, não horas: esforço, complexidade e incerteza juntos.

Sequência	1 · 2 · 3 · 5 · 8 · 13 · 21
Por que Fibonacci	Os intervalos crescem porque a precisão cai com o tamanho
Item de 21	Grande demais; quebre antes de planejar

Velocidade é quantos pontos a equipe conclui por sprint. Serve para **previsão**, medida a partir da terceira sprint.

> Velocidade não é produtividade e não compara equipes. Usar velocidade como meta faz a equipe inflar estimativas, e a métrica morre.

Por que não estimar em horas

Estimativa em horas	Estimativa relativa
Varia conforme quem executa	Independe de quem executa
Convida à comparação entre pessoas	Compara itens entre si
Sugere uma precisão que não existe	Assume a imprecisão
Exige reestimar quando a equipe muda	Continua válida

Tarefas do Sprint Backlog, por serem de curtíssimo prazo, costumam ser a exceção: ali as horas funcionam razoavelmente.

Planning poker

```
1. O Product Owner apresenta a história
2. A equipe pergunta até entender
3. Todos escolhem uma carta, ao mesmo tempo
4. Revelam juntos
5. Divergência grande? Quem votou mais alto e mais baixo explicam
6. Nova rodada, até convergir
```

O valor está no **passo 5**, não no número final.

> Quando alguém vota 2 e outro vota 13, os dois estão falando de histórias diferentes. Descobrir isso agora custa cinco minutos; descobrir na sprint custa a sprint.

Sprint Backlog

- Cada pessoa **escolhe** o trabalho que fará; não há atribuição
- A estimativa do trabalho restante é atualizada diariamente
- Qualquer membro pode acrescentar, alterar ou remover tarefas
- Tarefas pouco claras entram com estimativa maior e são subdivididas depois

É o único artefato que pertence exclusivamente aos Developers. O Product Owner ordena o Product Backlog; o Sprint Backlog é da equipe.

Definição de Pronto

Condições que **toda** história precisa cumprir para sair do quadro. Vale para todas, e não muda de item para item.

```
Definição de Pronto da equipe

  [ ] Código integrado na branch principal
  [ ] Revisado por outro integrante em pull request
  [ ] Testes automatizados escritos e passando
  [ ] Pipeline de CI verde
  [ ] Critérios de aceitação verificados
```

> "Está bom" não é definição. Definição de Pronto é uma lista que qualquer pessoa da equipe consegue conferir e chegar à mesma conclusão.

Pronto e aceito não são a mesma coisa

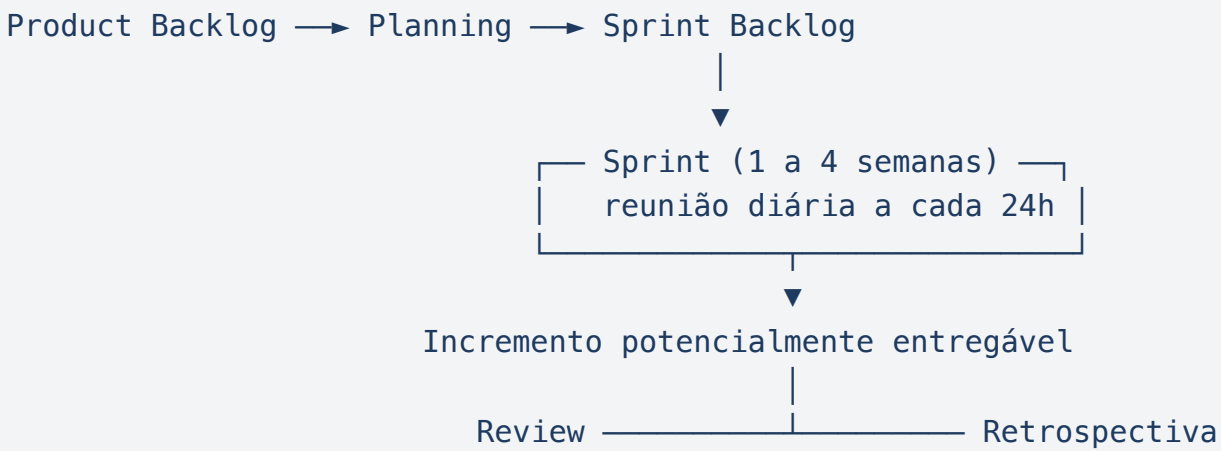
	Aplica-se a	Definido por
Definição de Pronto	**Todos** os itens	A equipe
Critérios de aceitação	**Aquele** item	O Product Owner

Um item está completo quando satisfaz **os dois**.

Os critérios de aceitação são **adicionais** à Definição de Pronto, e não substitutos dela. Um item pode passar em todos os seus critérios específicos e ainda assim não estar pronto, por não ter sido revisado ou por ter deixado a CI vermelha.

> A Definição de Pronto também **evolui**: uma equipe que ainda não tem ambiente de implantação não pode exigir "no ar em produção" hoje, mas pode acrescentar isso mais adiante.

O ciclo completo



Inspecionar e adaptar, em três escalas: a cada dia, a cada sprint e a cada revisão do backlog.

Simulação: Sprint Planning

Em grupo, com o projeto de vocês, por 15 minutos:

1. Enunciem um **Sprint Goal** em uma frase
2. Escolham **3 a 5 histórias** que sirvam a esse objetivo
3. Escrevam **critérios de aceitação** para a mais importante
4. Estimem duas delas em pontos, com planning poker
5. Escrevam a **Definição de Pronto** do grupo

> No fim, cada grupo lê o Sprint Goal em voz alta. Se a frase não deixar claro o que muda para o usuário, ela ainda não é um objetivo.

Parte 6

Próximos passos

Formação dos grupos

- **1 a 4 integrantes.** Cinco só com justificativa aprovada
- Formação até a entrega da Sprint 0
- Alterações valem a partir da sprint seguinte

Registrem hoje, no `README.md` do repositório: nome do grupo, integrantes com matrícula, e o usuário GitHub de cada um.

> Quem ainda não tem grupo: usem o canal do Discord.

O quadro no GitHub Projects

```
  Backlog  |  A fazer  |  Em progresso  |  Em revisão  |  Pronto
           |           |   WIP máx: 2   |  WIP máx: 2  |
```

1. Criem um **Project** vinculado ao repositório
2. Configurem as colunas acima
3. Definam o **limite de trabalho em progresso** por coluna
4. Criem as issues das histórias de hoje e vinculem ao quadro

> O quadro precisa refletir a realidade. Quadro atualizado na véspera da entrega é o oposto de transparência — e a atividade do repositório compõe 30% da nota da sprint.

Campos além do Status

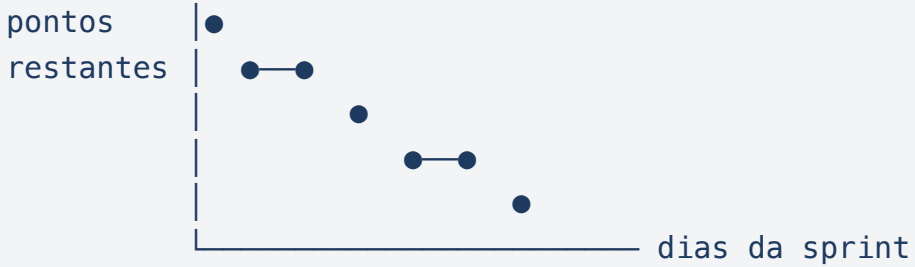
Poucos campos, cada um com uma finalidade clara:

Campo	Valores	Por que existe
`Tipo`	História · Tarefa · Defeito · Melhoria · Conhecimento	Muda a Definição de Pronto
`Pontos`	1 · 2 · 3 · 5 · 8 · 13	Estimativa relativa
`Sprint`	Iteração	Alimenta o burndown
`Risco`	Baixo · Alto	Ver abaixo

`Risco: Alto` indica que ainda **não se sabe se a abordagem funciona**, e não que o item seja difícil. É o tratamento explícito para itens de aquisição de conhecimento.

> Uma regra que ajuda: em toda sprint, o primeiro item puxado é o de maior risco.

Acompanhar a sprint



O gráfico de burndown mostra o **trabalho restante**, atualizado ao longo do ciclo. O GitHub Projects o gera a partir do campo de estimativa, na visão **Insights**.

Uma linha que sobe não é necessariamente má notícia: pode indicar que a equipe descobriu trabalho não previsto — informação útil, e melhor conhecida cedo.

> O gráfico serve para a equipe conversar sobre a sprint, e não para comparar pessoas.

Para segunda, 31/8

Tarefas

- Fechar o grupo e registrar no `README.md` do repositório
- Criar o repositório **público** e o quadro no GitHub Projects
- Escrever a visão do produto, usando o template de `docs/SPRINT-0.md`
- Trazer o backlog inicial rascunhado, com ao menos 10 itens

Leitura

- Manifesto Ágil · `agilemanifesto.org`
- Guia do Scrum · `scrumguides.org`

> Quarta é encontro **online**, no horário da aula, para dúvidas sobre a proposta e o acordo de processo.

Referências da aula

Da aula

- Beck et al. (2001) — *Manifesto para o Desenvolvimento Ágil de Software*
- Schwaber e Sutherland — *The Scrum Guide*, edição de 2020
- Cohn, Mike — *User Stories Applied*

Origem do Scrum

- Takeuchi e Nonaka (1986) — *The New New Product Development Game*
- Rubin, Kenneth — *Essential Scrum*

Livro-texto aberto

- Valente, *Engenharia de Software Moderna* · `engsoftmoderna.info`

Disciplina

- `github.com/fmarquesfilho/processos-2026-2`